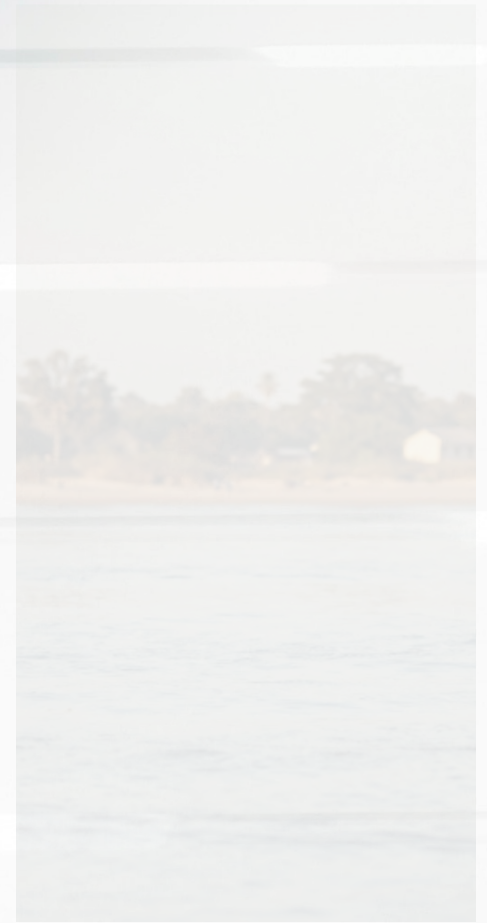
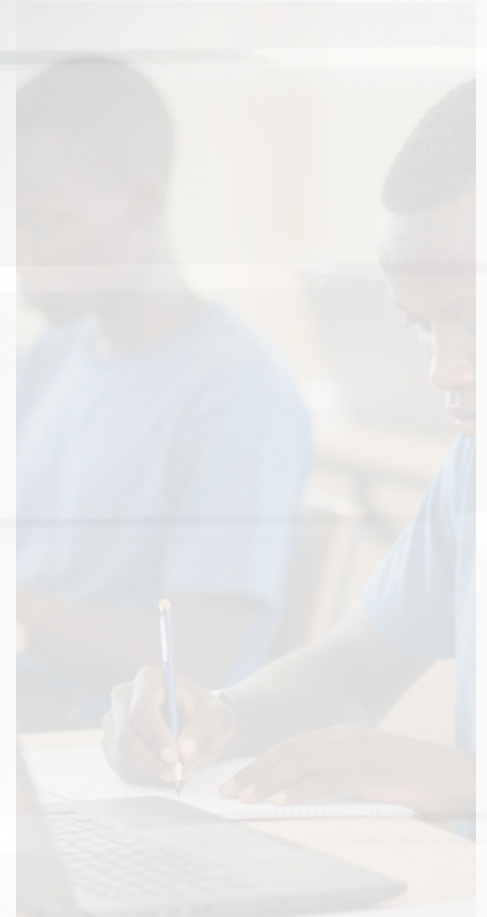


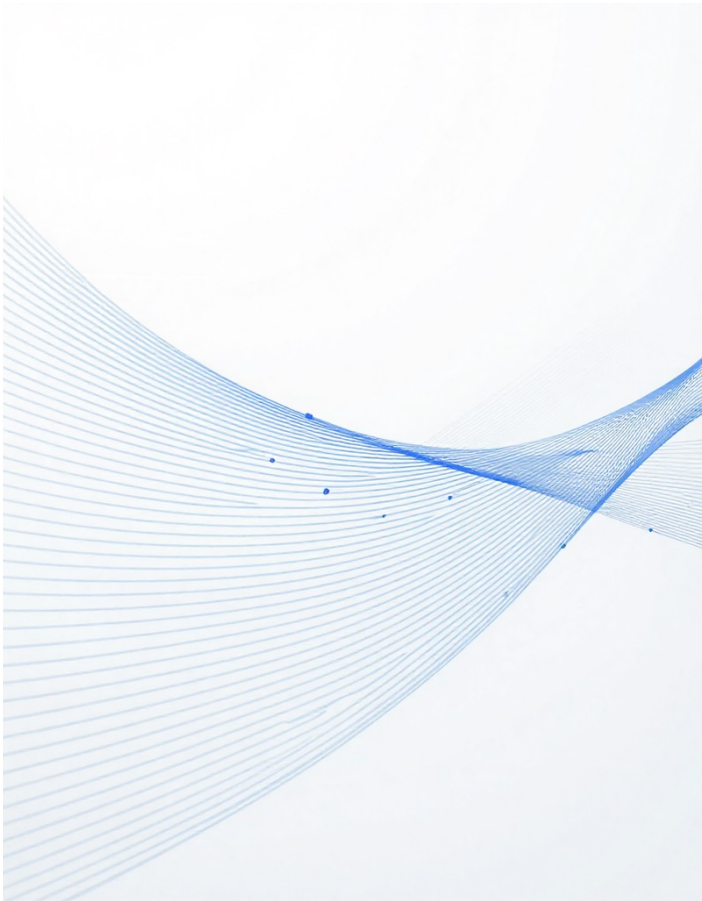
SDS 711: Programming for Data Science

- Bubacarr Bah, PhD,
Associate Professor and Head of Data Science,
MRC Unit The Gambia at LSHTM



Lecture 1: Introduction to Programming Concepts and Computational Thinking

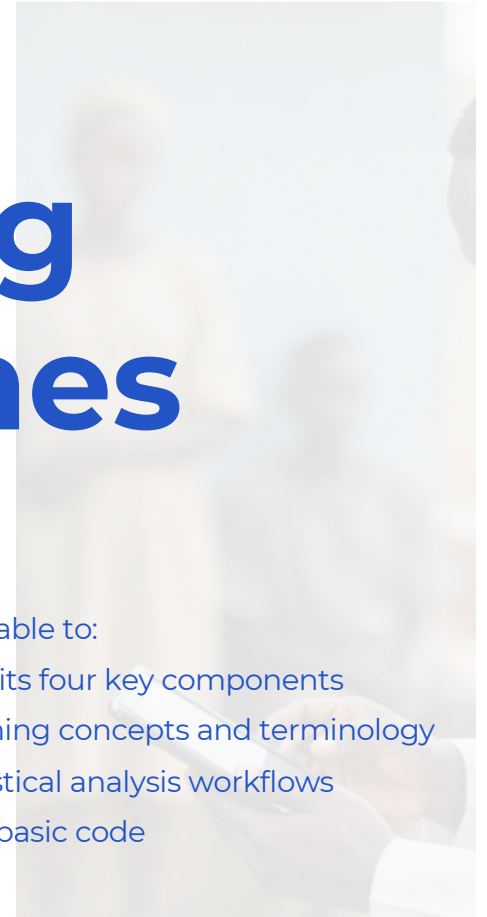




Learning Outcomes

By the end of this session, you will be able to:

- Explain computational thinking and its four key components
- Understand fundamental programming concepts and terminology
- Connect programming skills to statistical analysis workflows
- Design simple algorithms and write basic code

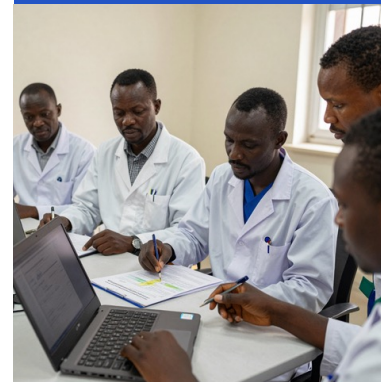


Why Programming?

Programming is a critical skill for modern statisticians. It enables reproducible research and automation, facilitates data cleaning and exploratory analysis, supports statistical modelling and machine learning, and powers data visualization and reporting.

Reproducible Research

Automation enables consistent, repeatable workflows. Code documents your analytical process and allows others to verify your results.

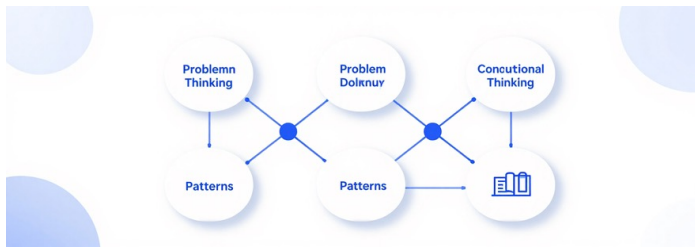


Data Science Workflow

From raw data to insights: cleaning, modelling, and visualization all depend on programming skills.



What is Computational Thinking?



A problem-solving process that formulates problems and solutions so a computer can execute them. Think about how you solve complex problems — do you use these steps?

Decomposition

Breaking down complex problems into smaller, more manageable parts that can be solved individually.

Abstraction

Focusing on critical information while ignoring irrelevant details to simplify the problem.

Pattern Recognition

Identifying similarities, trends, and regularities in data to find efficient solutions.

Algorithm Design

Creating step-by-step procedures to solve the problem systematically.

Decomposition Example: Malaria Surveillance Data

Step 1-2

Import data from sources and clean data to handle missing or inconsistent values.

Step 3

Explore variables through summary statistics and visual plots to understand the data.

Step 4-5

Build a statistical model to understand disease patterns and visualize results for stakeholders.



Programming Building Blocks



These are the fundamental pieces you'll use to write any program. Understanding these core concepts enables you to build complex solutions from simple components.

Variables & Data Types

Variables store data values. Data types include numbers, strings, and booleans that determine how data is processed.

Operators

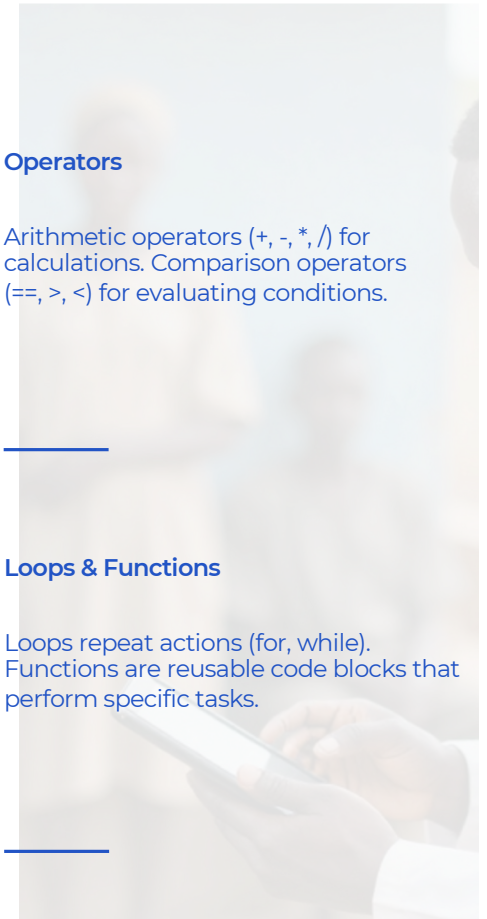
Arithmetic operators (+, -, *, /) for calculations. Comparison operators (==, >, <) for evaluating conditions.

Conditionals

If-else statements enable decision making in code. They allow programs to take different actions based on conditions.

Loops & Functions

Loops repeat actions (for, while). Functions are reusable code blocks that perform specific tasks.



Programming and Statistical Problem Solving

Programming connects seamlessly to statistical workflows through a simple framework:

Input → Processing → Output. This classic structure underpins how we transform raw data into meaningful insights. Understanding this flow is essential for effective statistical analysis and data science work.

Input

Collect raw exam scores from multiple classes and time periods.

Processing

Calculate average, standard deviation, and identify outliers.

Output

Generate summary statistics and visual reports for stakeholders.





Python Exercise: Calculate Average Score

```
scores = [65, 72, 88, 91, 75]
average = sum(scores) / len(scores)
print("Average score:", average)
```

Discussion Questions:

- What is the input?
- What processing occurs?
- What is the output?
- What changes if we have 5,000 students?

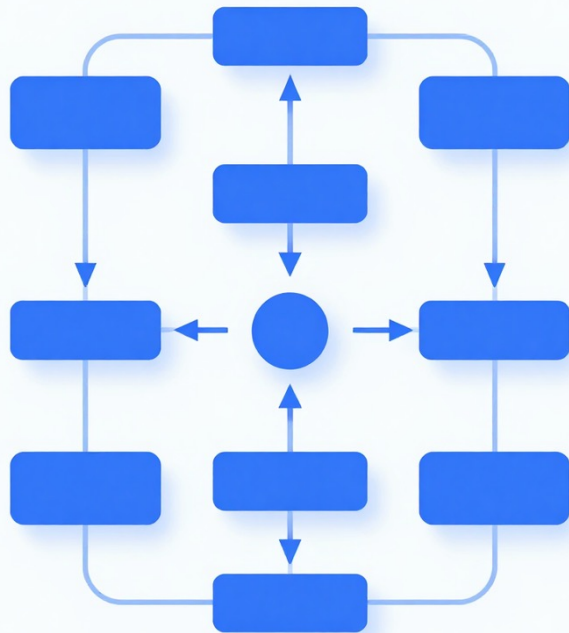


Python Exercise: Calculate Average Score

```
scores = [65, 72, 88, 91, 75]
average = sum(scores) / len(scores)
print("Average score:", average)
```

Discussion Questions:

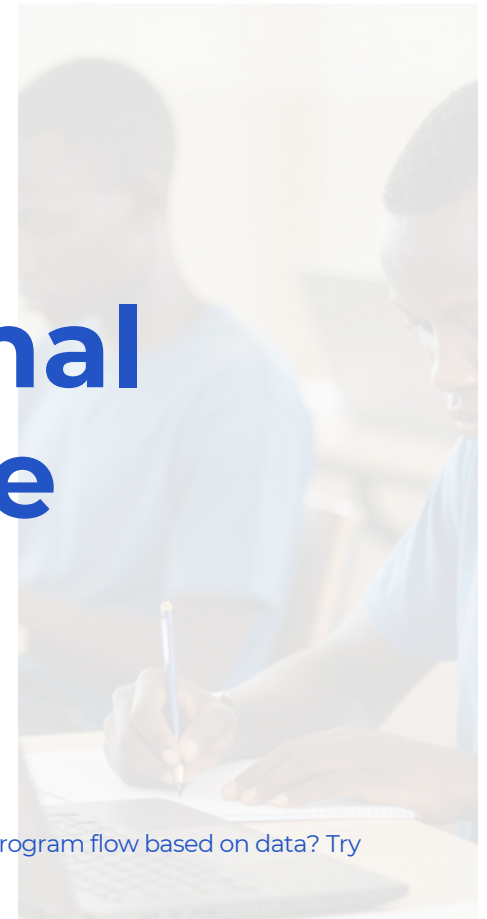
- What is the input? (the list of scores)
- What processing occurs? (sum and length, division)
- What is the output? (average printed to screen)
- What changes if we have 5,000 students?



Python Conditional Challenge

```
score = 72
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Discussion: How does this conditional control program flow based on data? Try changing the score value to 45, 50, and 80.





Mini R Exercise: Basic Statistics

R Code Example:

```
scores <- c(65, 72, 88, 91, 75)
```

```
mean(scores)
```

```
max(scores)
```

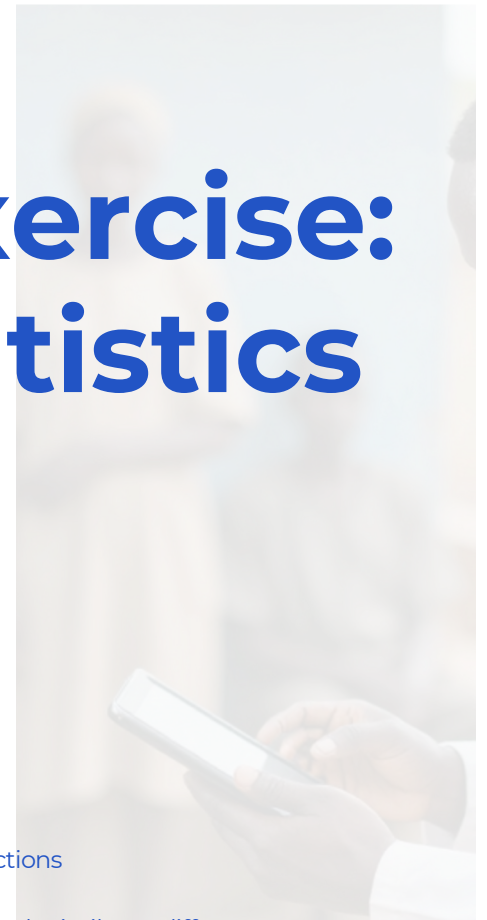
```
min(scores)
```

Explanation:

- `scores <- c(...)` creates a vector (vector creation)
- `mean()`, `max()`, `min()` are built-in summary functions

Teaching Prompt:

"Compare this to the Python example — what feels similar or different?"



Algorithms and Pseudocode

Pseudocode is a language-agnostic way to plan your solution before writing actual code. It helps you think through the logic clearly, making debugging easier and ensuring your algorithm works correctly before implementation.

Why Use It?

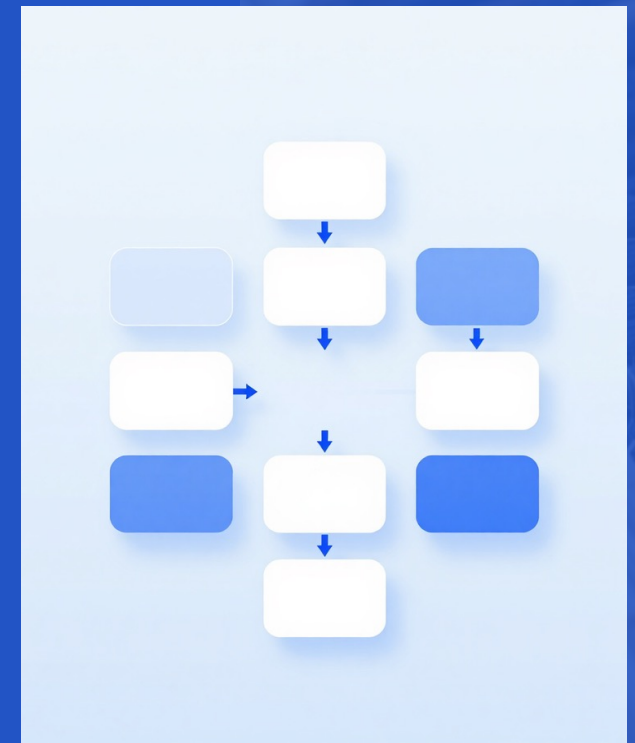
Plan the logic step-by-step without worrying about specific syntax or language rules.

Benefits

Communicate ideas clearly to others and focus on problem-solving over implementation details.

Your Turn

Try writing pseudocode for a median calculation: sort data, find middle value, return result. This exercise reinforces algorithmic thinking.

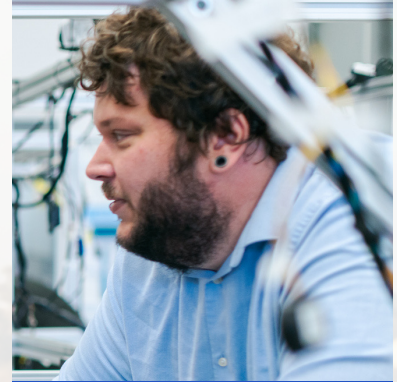
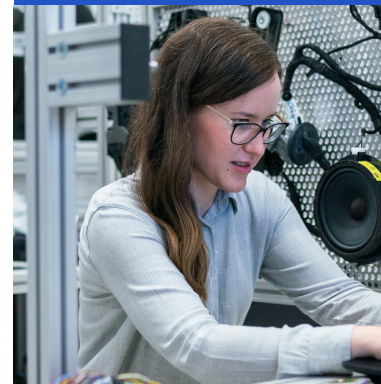


Activity Break: Design Your Algorithm

Take 5 minutes to write pseudocode for calculating the maximum score from a list of student grades. Use the scaffold below to guide your thinking, then share your approach with a neighbor.

Step-by-Step Scaffold (pseudocode)

1. Read the list of scores
2. Initialize max_score with first value
3. Loop through each score
4. If score > max_score, update max_score
5. Return max_score



Share & Discuss

Challenge: Can you write your pseudocode without looking at the scaffold? Then compare approaches with your group.

Key Takeaways



Programming is a versatile tool for solving statistical problems. Computational thinking guides problem formulation before coding. Algorithms are the bridge from problem to program.

Programming

Enables reproducible research, automation, data cleaning, and powerful statistical workflows for modern data science.

Languages

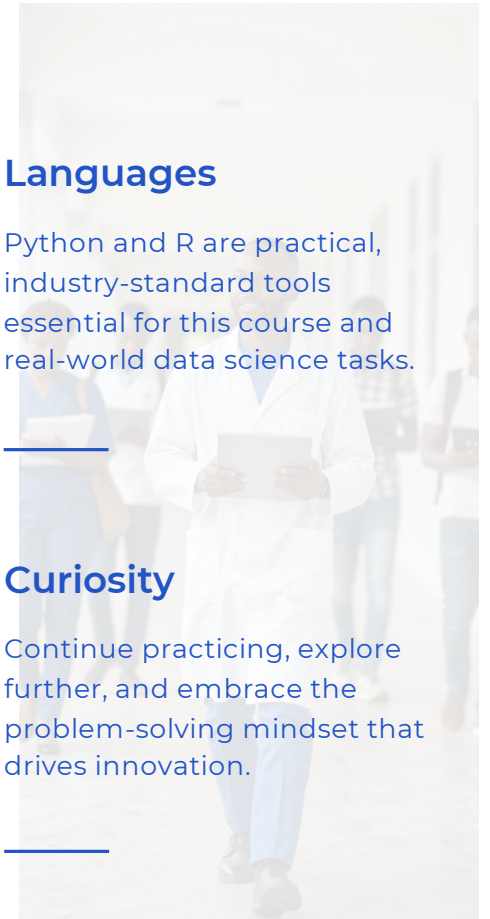
Python and R are practical, industry-standard tools essential for this course and real-world data science tasks.

ALGORITHMS

Bridge the gap between problem statements and executable code through structured, step-by-step thinking.

Curiosity

Continue practicing, explore further, and embrace the problem-solving mindset that drives innovation.



Questions & Discussion

Thank you for your attention! Any questions or thoughts?
Don't hesitate to reach out during office hours or via email. Review your exercises and prepare for the next session on Python and R syntax!

We're here to help you succeed



Office Hours: Tuesdays 2-4 PM, Room 204



stat.data@university.edu



www.university.edu/msc-stats

